

Automate FW testing with MDBCore scripting, part of the MPLAB®X ecosystem (module3a: automate testing with mdb)

Presenter Stefan Luethin

Nov '23

Agenda

1: what is the *MPLABX* ecosystem *MDBCore*, *SDK* -> how to use for automated FW testing

2: compile without IDE

3: Automated *FW-testing* with different *MDBCore* use-models

- a. simple *mdb* -> DOS/bash-script, easy to use but has limits
- b. more advanced *mdbcs* -> *.jar module needs scripting-language with Java-interface, more complex but solves *mdb* limits and opens door into Java-universe
- c. full, but most complex usage of *MDBCore* with Java -> SDK

4: summary

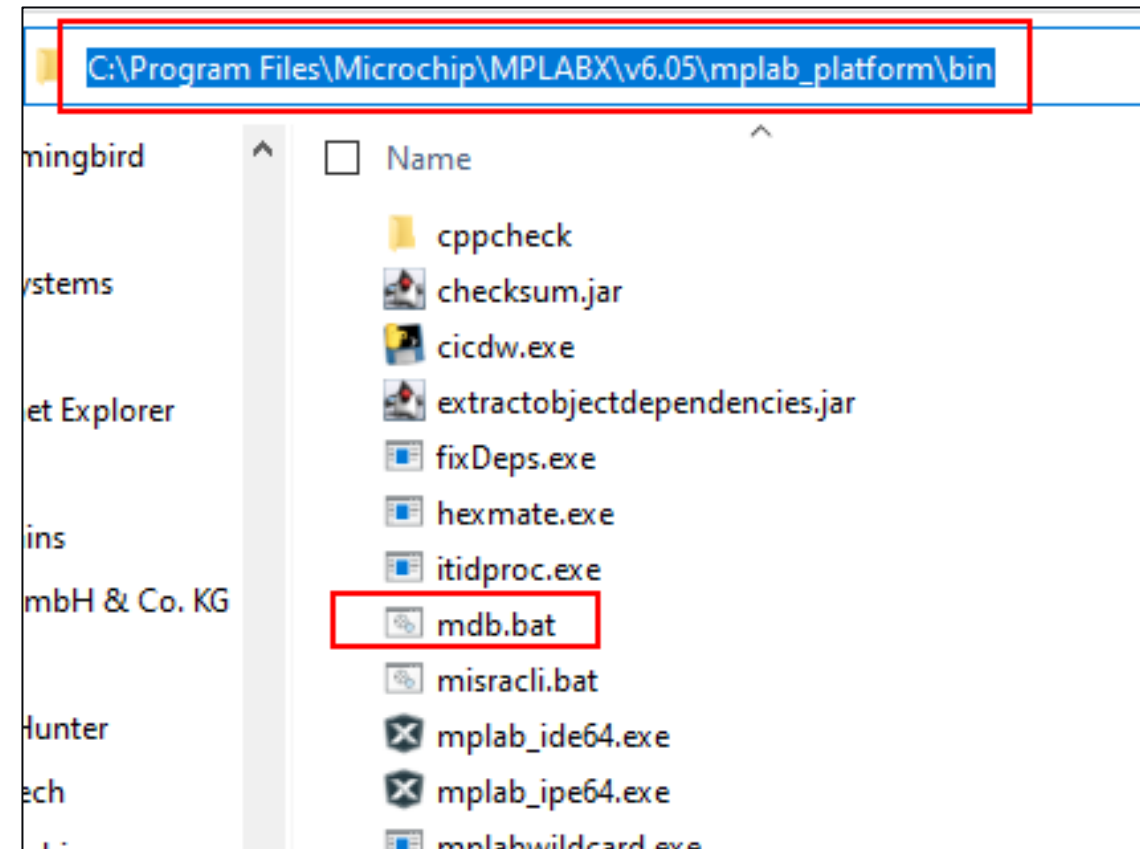
M3: overview

This section discusses the easiest MDBCore interface `mdb`

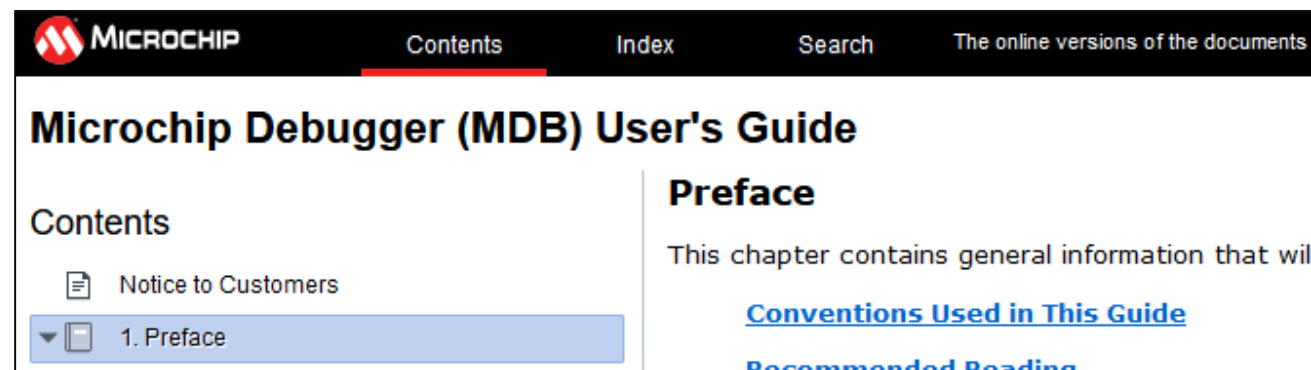
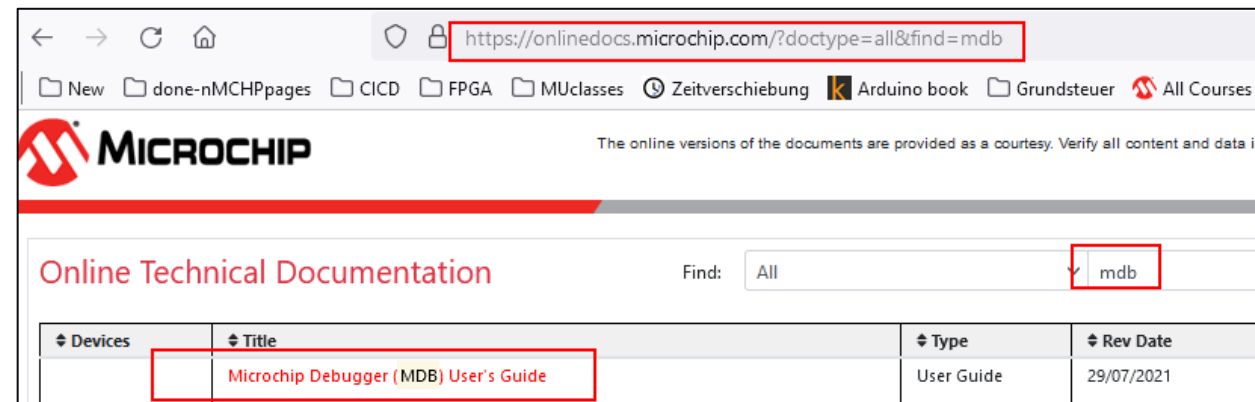
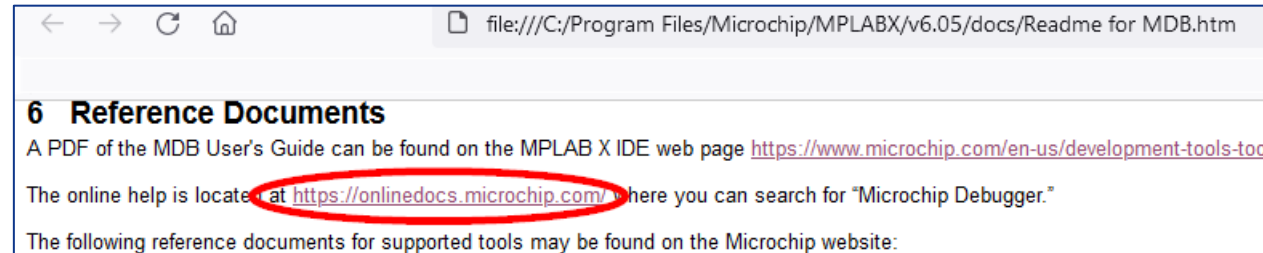
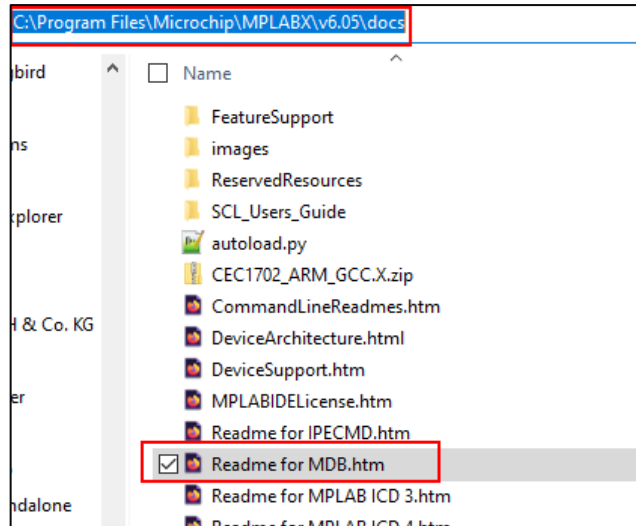
- What is `mdb` ?
 - Is it documented ?
 - What to install ?
 - Features and limits of `mdb` ?
-
- How does the `mdb` flow look like in general ?
 - What use models of `mdb` exist ?
 - A potential FW-regression flow using `mdb` looks like ...

M3: what is mdb?

- **mdb** stands for MicrochipDeBugger and is a MDBCore module
- It provides a **command line interface** for programming or debugging devices
- The command line interface makes it very easy to use on one hand **but** introduces limits on the other
- anything to install ?
 - > No extra installation as it comes with every MPLAB®X



M3: documentation for mdb?



M3: documentation for mdb?

 **MICROCHIP**

ContentsIndexSearch

The online versions of the documents are provided as a courtesy. Verify all content and data in the device's PDF documentation found on the device product page.

Microchip Debugger (MDB) User's Guide

Contents

- Notice to Customers
- 1. Preface
- 2. Introduction
 - 2.1. Installation and Documentation
 - 2.2. Getting Started
 - 2.2.1. Command Line Parameters
 - 2.2.2. Invoking the MDB
 - 2.3. Debugging Methods
 - 2.3.1. Entering Commands Method
 - 2.3.2. Programming a Production Image for T
 - 2.3.3. Debugging a Device
 - 2.4. Running a Command File Method
 - 2.5. Using Multiple Instances of the MDB
- 3. MDB Reference
 - 3.1. Help Commands
 - 3.2. Classes of Commands
 - 3.3. List of commands Within Classes
 - 3.3.1. Breakpoint Commands
 - 3.3.2. Data Commands
 - 3.3.3. Device and Tool Commands
 - 3.3.4. Others Commands
 - 3.3.5. Programming Commands
 - 3.3.6. Running Commands
 - 3.3.7. Stack Commands
- 4. Revision History
- The Microchip Website
- Product Change Notification Service

Breakpoint Commands

To display information about the breakpoints commands available in MDB, type help breakpoints. The following table provides additional information for this class of commands

Command	Description
Break	Sets a breakpoint at the specified source line number. Command format: break filename:linenumber [passCount] Example: break newmain.c:142 4 Sets a breakpoint at an absolute address. Command format: break *address [passCount] - address – the address of the program memory to break on – use the command: 'print /a' to get a symbol address. - passCount – the parameter is optional. Indicates the number of times the break 'on condition' is met before the program halts. Example: break *0x9d0000cc 5 MDB assigns a breakpoint number and returns:Breakpoint 0 at 0x9d0000cc: file newmain.c, line 16. Sets a breakpoint at the beginning of the function. Command format: break function name [passCount] Example: break function_foo 5
Delete	Deletes a breakpoint – if no argument is specified, this deletes all breakpoints. You can abbreviate this command as d. Command format: delete [breakpoint number] d [breakpoint number] The breakpoint number is generated by MDB for the Break and/or Watch commands. Examples: delete or D delete 1 or d 1
Halt	Stops the debugger program. Sets a data breakpoint at the specified memory address, variable name, or an SFR (special function register). Command format: Watch address breakonType[:value] [passCount]

M3: features of mdb?

- Supported features by `mdb` or in the `mdb` commandFile = command interface
 - Connect to debugger -> `hwtool`
 - Download *.elf or *.hex -> `program`
 - Data-/Variable-access
 - `Print` (`==read`)
 - `write` ->watch out to use the correct memory-location (->see example later one)
 - Execution control
 - `run, halt, next, wait, quit`
 - `breakpoint, stack`
 - Various properties
 - `set communication.interface swd`
- **Not supported / limits** (->later slide)

M3: features of mdb and use models?

best way to start with **mdb**

- check User'sGuide

OR

- start it interactively and

> help

or

> help command

```
C:\Windows\System32\cmd.exe - "C:\Program Files\Microchip\MPLABX\v5.45\mplab_platform\bin\mdb.bat"

C:\mchp\Masters\2021_eMasters-MU\24024DEV11_eMasters\labs\lab1_mdbbat>"C:\Program Files\Microchip\MPLABX\v5.45\mplab_platform\bin\mdb.bat"
com.microchip.crownking.mplabinfo 2021-05-12T17:31Z WARNING Device family is unknown for the device ATSAMRH71F208. Possibly DCore information is not mapped
com.microchip.crownking.mplabinfo 2021-05-12T17:31Z WARNING Device family is unknown for the device ATtiny3214. Possibly DCore information is not mapped
com.microchip.crownking.mplabinfo 2021-05-12T17:31Z WARNING Device family is unknown for the device EEC1005_SX1_UB1. Possibly DCore information is not mapped
com.microchip.crownking.mplabinfo 2021-05-12T17:31Z WARNING Device family is unknown for the device EEC1005_WC_UB1. Possibly DCore information is not mapped
com.microchip.crownking.mplabinfo 2021-05-12T17:31Z WARNING Device family is unknown for the device MEC1503. Possibly DCore information is not mapped
>help

The list of classes of commands:

breakpoints -- Making program stop at certain points
data -- Examining/Changing data
deviceandtool -- Selecting debug tool and device
others -- Others
programming -- Programming device and its relative functions
running -- Running the program
stack -- Examining stack

Type "help" followed by a class name for a list of commands in that class.
Type "help" followed by command name for full documentation.
Type "help" followed by "doc" to see how to open MDB documentation.

>help deviceandtool

List of commands:

Device -- Device devicename
Set the name of a target device.

Hwtool -- Hwtool [toolType] [-p] [index]
Set the debug tool, or list all available hardware tool on the system.
Use the -p option to connect to the tool for programming only.
Use the index option to select the tool if there is more than one instance of a tool type.
Use command hwtool to find the index number of the tool.

Use command "hwtool supported" (no quotation mark in your input command) to see all the supported tool types.

hwtool supported
List of supported tooltype:
licensedprogrammer (for Licensed Debugger)
atmelice (for Atmel-ICE)
edbg (for EDBG)
medbg (for mEDBG)
pkobnano (for PKOB nano)
powerdebugger (for Power Debugger)
jtagice3 (for JTAGICE3)
icd3 (for ICD 3)
pickit3 (for PICKit3)
sk (for Starter Kits (PKOB))
licenseddebugger (for Licensed Debugger)
realice (for Real ICE)
icd4 (for ICD 4)
pm3 (for PM3)
sim (for Simulator)
mchv (for MCHV)
microstick (for MICRSTICK)
skde33audio (for SKDE 33 AUDIO)
skdememory (for SKDE Memory)
skdepic18fj (for SKDE PIC18FJ)
skdepic24f1 (for SKDE PIC24F 1)
skdepic24hsensor (for SKDE PIC24H SENSOR)
skdepic32 (for SKDE PIC32)
pickit4 (for PICKit 4)
pkob4 (for Curiosity/Starter Kits (PKOB4))
snap (for Snap)
j32 (for J-32)
jlink (for J-Link)
seggersamice (for SEGGER SAM-ICE)
```

- **mdb** use-models (add `<mplabx>\mplab_platform\bin` to your OS-searchpath !)

- **interactively** -> just run `(dos)> mdb.bat`

- **automatically** -> store mdb-commands in commandFile and call `(dos)> mdb.bat cmdFile.txt`

M3: interactive usage (=principal flow mdb)

one mdb-test (regression will loop here)

*Develop your FW and compile into *.elf*

start mdb to get to mdb-prompt ">" (best to add path to OS-searchpath)

(DOS) >"C:\Program Files\Microchip\MPLABX\v6.15\mplab_platform\bin\mdb.bat"

At the mdb-prompt > select your device and set the debugger

> device ATSAME70Q21B

> hwttool EDBG #- '>hwttool supported' will list all known dbg

At the mdb-prompt > program/download your FW.elf (hierarchy: windows '\' - linux '/')

> program <path>\<prj>.X\dist\<config>\debug\fwDev11_same70_xult.X.debug.elf

At mdb-prompt > control execution – eg with break or halt

> break endOfTest

At the mdb-prompt > (reset and) execute/run

> run #- continue, step, ..

'wait' for some time (in msec) OR wait for breakpoint to occur and then 'halt'

> wait 3000

> halt #-!!important to 'halt' otherwise get cached data!!

At the mdb-prompt > access registers/data and examine memory

> Print testResult #-read/print global variable 'testResult'

> write /p 0xad 0x55 #-writes val=0x55 to address=0xad in program-mem

> x /p50x 0xad #-dump 50B of program memo in hex starting at 0xad

Finally quit mdb

> quit

(DOS) >

one ***pitfall***: need to 'halt' before using 'print' ->don't like this?

Well the SDK contains the NB-prj, so create our own mdb !

M3: automate mdb using command files

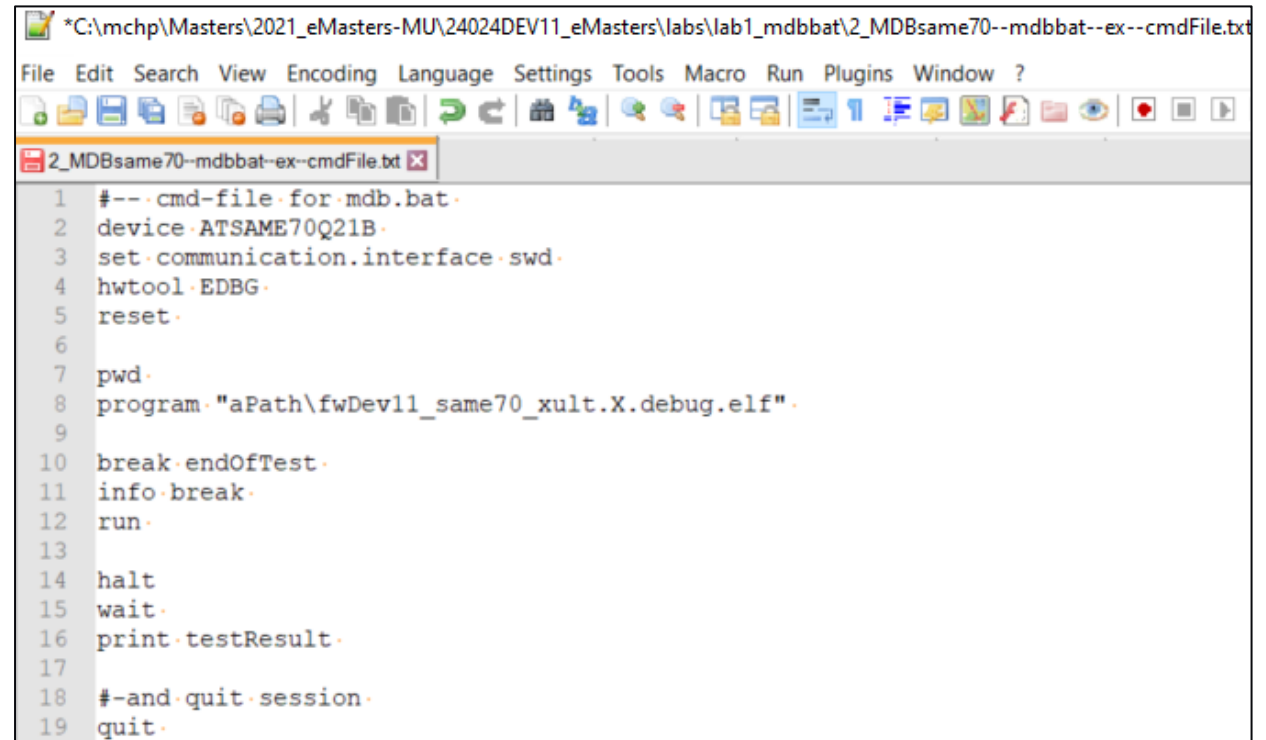
- Instead of entering `mdb`-commands interactively at `mdb` prompt, use `mdb`-command files
- what is a `mdb`-command file ?
->just a text file with `mdb`-commands

- using command-files:

`() > mdb.bat mdb_command.txt`

- Comment-lines start with '#'

- Dedicated logfile possible -> add a '|tee mylog.txt' to the call OR change `mdb.bat` ('tee' available in <mplabx>\gnuBins\GnuWin32\bin\tee.exe **and** '|' is part of DOS)



The screenshot shows a text editor window titled "2_MDBsame70--mdbbat--ex--cmdFile.txt". The window contains a list of 19 lines of commands for the `mdb.bat` script. The commands are: 1. `--.cmd-file.for.mdb.bat.`, 2. `device.ATSAME70Q21B.`, 3. `set.communication.interface.swd.`, 4. `hwtool.EDBG.`, 5. `reset.`, 6. (blank), 7. `pwd.`, 8. `program."aPath\fwDev11_same70_xult.X.debug.elf".`, 9. (blank), 10. `break.endOfTest.`, 11. `info.break.`, 12. `run.`, 13. (blank), 14. `halt`, 15. `wait.`, 16. `print.testResult.`, 17. (blank), 18. `#-and.quit.session.`, 19. `quit.`

```
1  #--.cmd-file.for.mdb.bat.
2  device.ATSAME70Q21B.
3  set.communication.interface.swd.
4  hwtool.EDBG.
5  reset.
6
7  pwd.
8  program."aPath\fwDev11_same70_xult.X.debug.elf".
9
10 break.endOfTest.
11 info.break.
12 run.
13
14 halt
15 wait.
16 print.testResult.
17
18 #-and.quit.session.
19 quit.
```

M3: extending mdb functionality with Python

Change `mdb` with SDK NB java-project or maybe enough extend with Python (undocumented)

- undocumented `mdb`-cmdfile keywords
 - `LoadScript`
 - `Execute`
- simple example
 - `mdbbat_test.py` with two functions `printMsg()` and `checkReg()`
 - use in `mdb` command file '`mymdbcmds.txt`'

```
# Load Python script to extends mdb.bat
loadscript mdbbat_test.py

# Print start signature
execute printMsg('INFO: test start')

# Print value of registers 'WREG8'
execute checkReg('WREG8', 20)
```
 - start as before

```
(DOS)> mdb mymdbcmds.txt
```
 - what is '`deb.GetNamedReg()`'?

MDBcore-function 'GetNamedReg()' which you can access with MDBCore-object 'deb' -> SDK for documention on MDBCore-functions

```
1  #-Description: mdbbat_test.py to extend mdb.bat
2
3  # Every successful check increases both counters
4  # Failed checks only increase total counter
5  passed = 0
6  total = 0
7
8  def printMsg(msg):
9      print(msg)
10
11 # Check if reg value equals val
12 def checkReg(reg, val):
13     if (deb.GetNamedReg(reg) == val):
14         updateStats(1,1)
15     else:
16         updateStats(0,1)
17
18 #eof
```

fct 'updateStats()' not shown but basically update global variables 'passed, total'

M3: limits of mdb?

- limits of `mdb` ?
 - Looping constructs like `while` / `for`
 - Controls like `if-else`
- imagine a flow like in pseudo-code left
->!!will not work with `mdb.bat`!!
- 'not supported' means: it is not directly possible in `mdb_command.txt` BUT
 - extend with python-fcts explained
 - Or create your own `mdb.jar` from NB-java-project of SDK
 - Or handled in a calling script, though might get complex
- best solution: `mdbc` explained in next module

```
device PIC16LF1947
hwtool icd3          #- takes option [id] if multiple ICD3 are connected!
program test.elf
    #-one test can run multiple times AND at the end of every run certain infos
    #-have to be dumped, like a variable or a piece of memory. Assume you've
    #-a fct 'end_of_run_fct()' which is hit whenever one run is done. Once hit
    #-you want to exec some mdb.bat-commands and continue until you hit this
    #-'end_of_run_fct()' again and do this loop until a variable "endOfTest"
    #-becomes '=1' and then stop and quit mdb.bat. So in pseudo-code the
    #-sequence looks like this:
break end_of_run_fct #- bkpt indicates one round is done -> some mdb.bat cmds
continue
while (1){
    #- once end_of_run_fct bkpt is hit
    x /p50x 0xad      #- eg: dump some memory which contains the test-result
    set testend= `print endOfTestVar` #-dump FW-variable 'endOfTestVar'
    if (1==testend)
        break
    else
        continue
} #- end-while-loop
quit #- now quit mdb.bat
```

M3: a suitable test structure

Before diving into automated FW-testing, we first need to define a suitable test structure

- this is just a suggestion and the final implementation depends on your needs!
- A reasonable setup of variables (self-explanatory by their names):
 - `uint32_t testResult , testID , testTimeout;`
- If a FW-release should work on different board-revisions use a macro like `ALL_BOARDS, PCBv1,...`
- Surround test code with `#ifdef TEST` -> set macro `TEST` when running test
- while test is running, continuously check for
 - either `testTimeout` (so one hanging test would not stall whole regression)
 - or `testEnd` criteria (might just be a button-press)and jump to `endOfTest ()` (one common breakpoint for all tests!)
- In function `endOfTest ()` check (just to name a few bullets):
 - if `testTimeout` reached (eg: `actualTestTime == testTimeout`) or test ended with a result
 - create test result (eg: `average over memory[x-y] < threshold`) to decide test=pass/fail and set `testResult`
- Finally a regression would loop over several tests
- Clearly 'Testing' can be a complex task, hence professional tools (eg: `Jenkins`) or companies (eg: `LDRA`) are available, but that's beyond the scope of this class

M3: a potential mdb based regression flow

A regression.bat might look like this:

```
#!/bin/bash
#-variables for regression
regrName=<name-of-regression>
testRootPath=<path-to-all-tests>
# each one is a mdbbat-cmdfile
list_of_tests="t1.txt t2.txt ... T10.txt"
#-variables for mdb-command-files
testDevice=ATSAME70Q21B
testDbg=EDBG
```

prep

```
##--MAIN--##
##---compile revision into *.elf---###
cd $revPath\${rev} #-contains $rev.X and Makefile
git checkout $rev #-checkout FW-rev $rev
prjMakefileGenerator.bat $rev
cd ${rev}.X
make
#->result will be in
'$rev\dist\production\${rev}.production.elf'
```

compile

```
##---start regression---###
echo "Start regression @`date`" | tee ${regrName}
#-loop over all test-cmdfiles
for test in $list_of_tests; do
    echo "Start $test @`date`" | tee -a ${regrName}
    #-create mdbbat-cmd file as mdbbat doesn't support vars
    echo "#-mdbbat-cmd-file for test $test" >> ${test}_mdb.txt
    echo "device $testDevice" >> ${test}_mdb.txt
    echo "hwtool $testDbg" >> ${test}_mdb.txt
    testElf=$testRootPath\${test}\dist\program\${test}.program.elf
    echo "program $testElf" >> ${test}_mdb.txt
    echo "break endOfTest" >> ${test}_mdb.txt
    echo "run" >> ${test}_mdb.txt
    echo "wait" >> ${test}_mdb.txt
    echo "print test_result" >> ${test}_mdb.txt
    #-need to convert 'test_memory' into hex-address
    testMemA=get_addr_of(test_memory)
    #-need to provide how much data of which typ to dump
    testMemSize=/p10dw
    echo "x $testMemSize ${testMemA}" >> ${test}_mdb.txt
    echo "quit" >> ${test}_mdb.txt
    #-finally start test $test
    call mdb.bat ${test}_mdb.txt 2>&1 | tee ${test}_run.log
    echo "Finished test $test @`date`" | tee -a ${regrName}
```

regression loop

test-creation

test-execution

done

```
##---start regression---###
```

M3: Lab for mod3/mdb

- There's a lab for this module that shows how to use `mdb` commandfiles
- Next slides show
 - first what you need to setup (tools, versions,...)
 - how to get the Lab material
 - and how to run the lab
 - Finally, the demo is run, so you see how it is supposed to work

M3: Lab for mod3/mdb -> how to get&run lab

Steps to get+run the example for this mod3/mdb

- Assumptions (tools, versions,..)
 - Install MPLABX-v6.05 and XC32-v4.21 (any other should work) in standard-paths
 - Extend OS-searchpath "`<mplabx>\mplab_platform\bin:<mplabx>\gnuBins\GnuWin32\bin`"
 - Need some version of `git` installed on your system OR you manually pull the repository
 - Required HW [atsamd21-xpro](#) and it is attached via USB to your host
 - Need any terminal program installed, like TerraTerm
- First clone repository from git locally with (!!--don't do this in a onedrv-folder, spaces in the path create problems--!!)

```
(DOS:...) > cd <your-directory-of-choice = myDir>
```

#-> open a DOS-console and use git to pull the repository

```
(DOS:... \myDir\)> git clone https://github.com/stefanluethin-microchip/mu_dev11_mdbcore
```

#->will create new directory '`mu_dev11_mdbcore\`' OR create directory and get repo manually

- Next, update Makefiles and compile, as explained in module 'compile without IDE':

```
(DOS:mu_dev11_mdbcore\)> prjMakefilesGenerator.bat .\dev11Labs_fw\firmware\CICDgit_samd21xplp.X
```

```
(DOS:mu_dev11_mdbcore\)> pushd . && cd .\dev11Labs_fw\firmware\CICDgit_samd21xplp.X && make && popd
```

- Go back to toplevel-directory (contains .git folder) and run the provided mdb-example with

```
(DOS:mu_dev11_mdbcore\)> .\dev11Labs_mdbScripts\dev11mod3Lab_mdbbat-1a-run.bat
```

- next slide briefly explains implemented functionality and then will see expected result

M3: Lab for mod3/mdb -> used FW

```
54 int main ( void )
55 {
56     /* Initialize all modules */
57     SYS_Initialize ( NULL );
58     printf("\n\r");
59     printf("\n\r START CICDtesting ...");
60
61     //SL: inits
62     myCnt=0; //SL: central counter
63     finalLedCnt=0;
64     sysTickTmrExFlag=false; //SL: SysTickWrap-event flag
65     //SL: SW0 defined as GPIO-In but still need to call this fct to enable In
66     SW0_InterruptEnable();
67     char2prntIdx=0; // default char-to-print = 'b'
68
69     SYSTICK_TimerStart();
70     SYSTICK_TimerCallbackSet(&sysTickTimeout_handler, (uintptr_t) NULL);
71
72     while ( true )
73     {
74         /* Maintain state machines of all polled MPLAB Harmony modules. */
75         if(sysTickTmrExFlag)
76         {
77             sysTickTmrExFlag=false;
78             LED0_Toggle();
79             finalLedCnt++;
80
81             myCnt++;
82             if(cEchoCnt/2>=myCnt)
83             {
84                 printf("a");
85             } else if ((cEchoCnt/2<myCnt) & (cEchoCnt==myCnt))
86             {
87                 //printf("b");
88                 printf("%c",charArray[char2prntIdx]);
89                 if (cEchoCnt==myCnt)
90                     myCnt=0;
91             } else
92                 printf("ERROR1");
93         }
94
95         if(SW0_Get() == PRESSED)
96         {
97             testEnd();
98         }
99     }
100
101     /* Execution should not come here during normal operation */
102
103     return ( EXIT_FAILURE );
104 }
```

```
37 volatile uint32_t myCnt;
38 volatile uint32_t finalLedCnt;
39 #define cEchoCnt 10
40 #define PRESSED 0
41 #define RELEASED 1
42 volatile bool sysTickTmrExFlag;
43
44 char charArray[3] = {'b','c','d'};
45 volatile uint32_t char2prntIdx; // index for 'charArray'
46 void testEnd(void);
47
48
49 void sysTickTimeout_handler(uintptr_t context)
50 {
51     sysTickTmrExFlag=true;
52 }
```

```
COM8:115200bps - Tera Term VT
File Edit Setup Control Window Help

START CICDtesting ...aaaaabbbbbbaaaabbbcccaaaaaacccccaaaa
aaacccccaaaaacccccaaaaacccccaaaaac
testEnd with finalLedCnt=116
```

```
106 void testEnd(void)
107 {
108     printf("\n\r testEnd with finalLedCnt=%d\n\r ",(int)finalLedCnt);
109     printf("\n\r");
110     while(1){}
111 }
```

Three FW functionalities:

1. LED blinks @SYSTICK=250ms
2. echo a char on uart-console
 - 1.char printed is always 'a'
 - 2.char selected from array `charArray` with variable `char2prntIdx`
 - print 5x1.char/'a' and 5x2.char/'b,c,d' (default 'b')=> changing `char2prntIdx` shows that you can change FW behaviour from the script
3. on HW-button SW0 press
 - it jumps to `endOfTest()`
 - in `endOfTest()` print the value of total LEDtoggles stored in `finalLedCnt`
 - and loop there

M3: Lab for mod3/mdb -> expected result

Demo now

```
C:\Windows\System32\cmd.exe
COM8:115200bps - Tera Term VT
File Edit Setup Control Window Help

START CICDtesting . .aaaaabbbbbbaaaabbcccccccaaccccccaaaaaccccccaaaaacc
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
testEnd with finalledCnt=116

C:\mchp\Masters\2021_eMasters-MU\24024DEV11_eMasters\eMaster_MU-version3\dev11labs\mu_dev11_mdbcore>
#####(dev11mod3Lab_mdbbat-1a-run.bat): Starting ...
#####: Starting dev11mod3Lab_mdbbat-1a-run.bat from C:\mchp\Masters\2021_eMasters-MU\24024DEV11_eMasters\eMaster_MU-version3\dev11labs\mu_dev11_mdbcore
##### Starting mdb.bat call: -mdb.bat ".\dev11labs_mdbScripts\dev11mod3Lab_mdbbat-1b-command.txt"-
/*Starting dev11mod3Lab_mdbbat automatic*/
device ATSAMD21J18A
set system.disableerrormsg true
set system.disableoutput true
set system.yestoalldialog true
set communication.interface swd
fwtool EDBG
Jan 10, 2023 6:26:30 PM org.openide.util.NbPreferences getPreferencesProvider
WARNING: NetBeans implementation of Preferences not found
Jan 10, 2023 6:26:38 PM com.microchip.mplab.mdbcore.RealICETool.RIMessages OutputMessage
INFO:
Program .\dev11labs_fw\firmware\CICDgit_samd21xplp.X\dist\samd21xplp\production\CICDgit_samd21xplp.X.production.elf
Program succeeded.
break testEnd
breakpoint 0 at function testEnd.
reset
run
/*start run for 4.500msec=4,5sec - stop at any time by pressing SW0*/
wait 4500
halt
/*- print myCnt = */
/*- print char2prntIdx -*/
/*- print char2prntIdx 0 #0->'b', 1->'c', 2->'d' -*/
print char2prntIdx
Stop at
    address:0x166c
    file:C:\mchp\Masters\2021_eMasters-MU\24024DEV11_eMasters\eMaster_MU-version3\dev11labs\mu_dev11_mdbcore\dev11labs_fw\firmware/src/main.c
    source line:95
>char2prntIdx=
0
print finalledCnt
finalledCnt=
17
/*change char2prntIdx=1 -> should now see aaaaacccccaaaa...*/
write /p char2prntIdx 1
continue
wait 3500
/*###-quit mdb... -> stop FW by 'pressing SW0'*/
quit
Stop at
    address:0x1670
    file:C:\mchp\Masters\2021_eMasters-MU\24024DEV11_eMasters\eMaster_MU-version3\dev11labs\mu_dev11_mdbcore\dev11labs_fw\firmware/src/main.c
    source line:75
>
=====dev11mod3Lab_mdbbat-1a-run.bat): SUCCESS
C:\mchp\Masters\2021_eMasters-MU\24024DEV11_eMasters\eMaster_MU-version3\dev11labs\mu_dev11_mdbcore>
```

Demo now

M3: mdb - Knowledge Check

Summary and knowledge check module3: using `mdb`

- what is `mdb` ?
=> simple MDBCore-IF using a command-file and provided via a java-call wrapped into a script
- where is the documentation for `mdb` ?
=> `<mplabx_install_path>/docs/Readme for MDB.html`
- how to use `mdb` ?
=> interactively OR automatic with `commandFile`
- features of `mdb` ?
=> `connect/hwtool -> program *.elf -> run,halt,wait, breakpoint -> print,write`
- limits of `mdb` ?
=> no looping, no if-else, cannot define user functions,... => next module 'mdbcs' shows how to solve them
- advantage of `mdb` ?
=> comes with every MPLABX-version, hence nothing to install
- how to extend `mdb` ?
=> either with hidden commands `LoadScript&execute`
=> or SDK provides NB-prj of `mdb`, so you can create your own

End Mod3: mdb

End of Mod3:

automated FW-testing using MDBCore-IF *mdb*

Next Mod4:

automated FW-testing using MDBCore-IF *mdbcs*